Verification Plan version 1

Group 8: Handwritten Digit Classification System Powered by the FPGA

Verification team (Group 10): Ashwin Karthik VK, Giridhar Ramesh

Design team (Group 8): Qing Mu, Marcel Martens, Melwin Bauer

Target grade: 4 (UVM on two cascaded blocks)

Target blocks: router_input_multi feeding cnn_integration_top

1 Introduction

The modules to be verified are the input router (router_input_multi) and one CNN accelerator core (cnn_integration_top) from Group 8's handwritten digit classifier. The router receives images over a 32-bit AXI-Stream, splits them into 784-pixel images and sends them one pixel at a time to the core. The core runs the CNN (convolution, max-pooling, flatten and a fully connected layer) and returns ten int8 class scores per image. For grade 3 the core is verified on its own. For grade 4 the router and the core are verified together.

Verification objectives

- Ensure each CNN core produces the correct ten class scores for every image, bit-exact against our reference model, including edge cases such as all-zero, all-255 and single-pixel images.
- Verify the input router splits the 32-bit stream into whole images, sends image N to core N mod 2, and delivers every pixel exactly once and in order.
- Check for correct handling of the ready/valid handshakes, input stalls, output backpressure, image and package boundaries, illegal inputs and reset.

2 Verification environment

The methodology used for this verification is UVM in simulation. Constrained-random tests, directed tests and SystemVerilog assertions (SVA) are combined to check the design. For grade 3 the DUT is one cnn_integration_top. For grade 4 it is router_input_multi built with two cores (NUM_ACCEL = 2), each core being a cnn_integration_top. The package size (PKG_SIZE), which normally comes from software, is set by the testbench.

Verification components UVM

- **Sequence item:** one 784-pixel image, with its image type (real, random, sparse, all-zero, all-255, impulse or pattern) and the delays to insert between transfers.
- **Sequence:** generates the images for the input side and the ready patterns for the output side. A top-level sequence runs both at the same time, so input traffic and backpressure are controlled from one test.
- **Sequencer:** passes sequence items from the sequence to the driver.

- **Driver:** drives the DUT pins. On the 32-bit AXI-Stream it packs four pixels per beat and sets tlast on the last beat of a package. On the 8-bit pixel port it sends one pixel per transfer. On the result ports it only drives ready.
- **Monitor:** watches one interface and turns pin activity into transactions for the scoreboard and coverage. There is a monitor on the DUT input, on each router-to-core link and on each result output.
- **Agent:** groups the sequencer, driver and monitor of one interface. Agents that drive signals are active. The agents on the router-to-core links only watch the signals, so they are passive.
- **Configuration:** configuration objects set the grade, whether each agent is active or passive, and the randomised test parameters (package size, gap lengths and backpressure). They are passed from the test down to each component through the UVM configuration database (uvm_config_db), so settings do not have to be hard-coded in the components.
- **Scoreboard:** receives transactions from the monitors. It checks that image N reaches core N mod 2 unchanged, and compares the ten scores from each core with the reference model.
- **Reference model:** a SystemVerilog model of the quantised CNN that computes the expected scores, compared bit-exact. Group 8's Python model is used as a cross-check, and as the reference until Group 8 provides the requantisation formula.
- **Coverage collector:** records which functional coverage points have been hit, using transactions from the monitors and values from the reference model.
- **Environment and test:** the environment builds and connects all components. Each test chooses the sequences and configuration to run.

3 Test scenarios

CNN core

- **Basic classification:** real MNIST images; the scores must match the reference model.
- **Random and sparse images:** pixel values weighted towards 0 and 255 with dist, to reach sums that real digits never produce.
- **Extreme images:** all-zero and all-255 images, to test accumulator overflow and clamping to int8.
- **Impulse images:** one bright pixel in a corner, on an edge and in the middle; shows window shifts and edge handling.
- **Patterns:** stripes and a checkerboard; shows swapped rows or columns and a wrong flatten order.
- **Back-to-back and repeated images:** the result for an image must not depend on the image before it.
- **Input stalls and output backpressure:** random gaps on the input and fc_out_ready held low; no data may be lost or repeated.
- **Latency:** with no stalls, the cycles from the last pixel to fc_out_valid must be the same for every image. The expected value still has to come from Group 8.

- **Reset mid-image:** after a reset during an image, the next image must still give the correct result.

Input router

- **Pixel unpacking:** four pixels leave the router per 32-bit beat. The byte order is checked once Group 8 defines it.
- **Framing and round-robin:** every 196 beats close one image, and image N goes to core N mod 2. Whether the order restarts at core 0 for each package is checked once Group 8 defines it.
- **Gaps and busy core:** idle cycles on tvalid, and one core stalled; the router must wait, not drop pixels or skip to the other core.
- **Package size and tlast:** package sizes from 1 to the largest the DMA allows, with tlast on beat 196 x package size.
- **Back-to-back packages and reset:** packages sent with no reset between them, and a reset in the middle of a package; the router must carry on correctly.
- **Illegal input:** early or late tlast, partial tkeep and a package size of 0. The spec leaves the expected behaviour undefined, so for now we only check that the DUT recovers after a reset.

Router and cores together

- **Parallel operation:** the busy periods of the two cores overlap for consecutive images.
- **Random stress:** all of the above randomised over many packages; this is the main regression test.

4 Coverage points

- **Image Types:** each image type: real MNIST, random, sparse, all-zero, all-255, impulse (corner, edge and middle), stripes and checkerboard.
- **Pixel Values:** bins for 0, 1 to 127, 128, 129 to 254 and 255. Pixel 128 gets its own bin because it becomes exactly 0 after the uint8 to int8 conversion.
- **Output Scores:** each of the ten scores seen at -128, negative, 0, positive and +127, and each class as the highest score at least once.
- **Stalls and Backpressure:** short and long input gaps and ready-low periods, at the start, middle and end of a row.
- **Package Sizes:** a single image, small and large packages and the largest allowed, with tlast on the last beat each time.
- **Core Assignment:** images sent to core 0 and core 1, including the first image of a new package, with each core busy while the other is next in turn.
- **Parallel Operation:** both cores busy in the same cycle, and backpressure on both cores at once.
- **Reset Cases:** reset during pixel input, during computation and while a result is waiting.
- **Assertions:** the cover property of each assertion is hit at least once.

5 Bug tracking

- If a bug is found it is reported in the coverage sheet in the Status column, with a short description in Comments.

6 Sign-off

- After all the test cases are implemented and pass, the verification process is done. The coverage sheet is subject to change at any moment throughout the verification process.